

# Poker Hand Prediction with XGBoost and Feature Engineering

Zehui Sun

2025-12-03

## Feature engineering

Here is how I manually engineer the feature.

Basically I use max rank, second largest rank, max suit count and number of unique counts and rank span as inputs. Because normal machine learning model cannot learn the basic logic of those rules (I suspect there is underrepresentative issues with these rules, i.e. those information is actually not enough to depict those rules). So I manually feed the feature into the model.

## Load the Best Model

The script automatically locates the most recently generated XGBoost model file (based on the timestamp in the filename).

```
latest_model_file <- "../saved_model/latest_model.model"
cat("Loading model:", latest_model_file, "\n")

## Loading model: ../saved_model/latest_model.model

# prepare test data
cols <- c("S1", "R1", "S2", "R2", "S3", "R3", "S4", "R4", "S5", "R5", "Class")
test <- read.csv("../data/poker-hand-testing.data", header = FALSE, col.names = cols)
X_test <- engineer_features(test)
y_test <- test$Class
X_test_a <- as.matrix(X_test)

# Create DMatrix for XGBoost
dtest <- xgb.DMatrix(data = X_test_a, label = y_test)

model <- xgb.load(latest_model_file)
pred_prob <- predict(model, dtest)
pred_mat <- matrix(pred_prob, ncol = 10, byrow = TRUE)
pred_lab <- max.col(pred_mat) - 1

conf_matrix <- confusionMatrix(
  factor(pred_lab, levels=0:9),
  factor(y_test, levels=0:9)
)
```

```
# acc
acc <- round(conf_matrix$overall["Accuracy"] * 100, 4)
cat("Overall accuracy on test:", acc, "%\n")
```

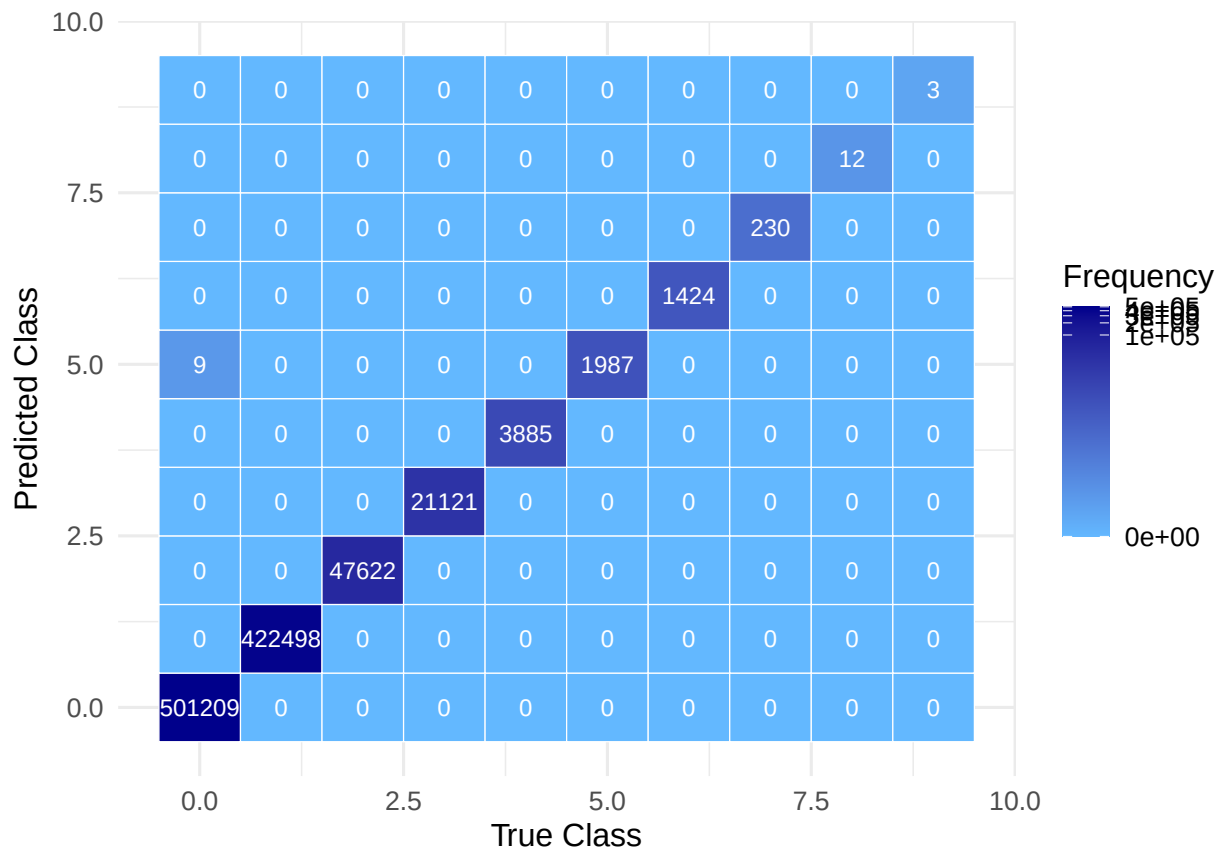
```
## Overall accuracy on test: 99.9991 %
```

Given the highly imbalanced nature of the dataset (where Class 0 constitutes ~50%), high accuracy alone does not guarantee good performance on rare hands. We must examine more results.

## Evaluation

```
mat <- as.matrix(conf_matrix$table)
df_plot <- melt(mat)
colnames(df_plot) <- c("True_Class", "Predicted_Class", "Frequency")

ggplot(df_plot, aes(x = True_Class, y = Predicted_Class, fill = Frequency)) +
  geom_tile(color = "white") +
  geom_text(aes(label = Frequency), color = "white", size = 3) +
  scale_fill_gradient(low = "steelblue1", high = "darkblue", trans = "log1p") +
  theme_minimal(base_size = 12) +
  labs(x = "True Class", y = "Predicted Class")
```



Print some metrics.

```
metrics_df <- conf_matrix$byClass[, c("Precision", "Recall", "F1")]
kable(metrics_df, digits = 4, caption = "Performance metrics for each class")
```

Table 1: Performance metrics for each class

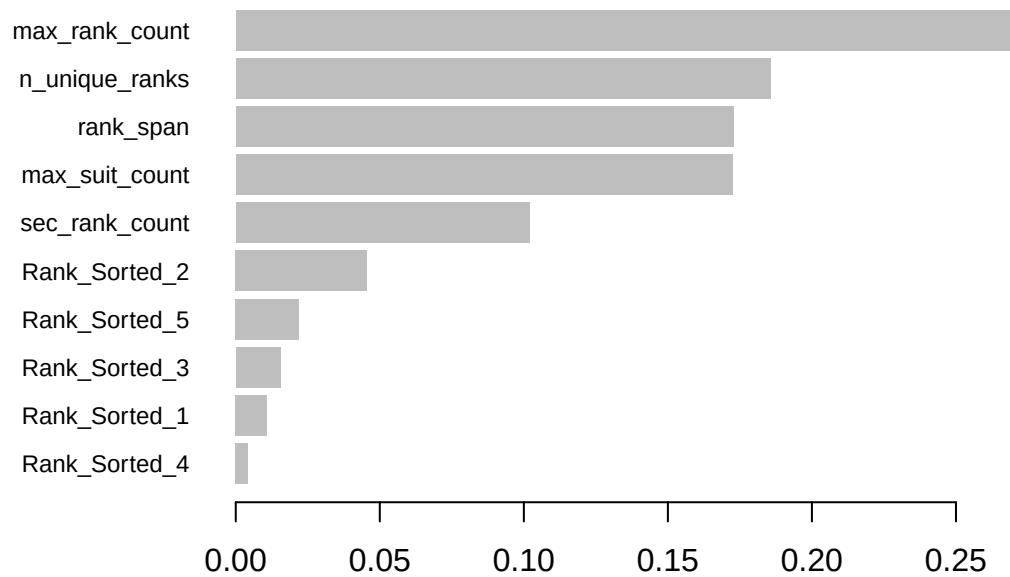
|          | Precision | Recall | F1     |
|----------|-----------|--------|--------|
| Class: 0 | 1         | 1.0000 | 1.0000 |
| Class: 1 | 1         | 1.0000 | 1.0000 |
| Class: 2 | 1         | 1.0000 | 1.0000 |
| Class: 3 | 1         | 1.0000 | 1.0000 |
| Class: 4 | 1         | 1.0000 | 1.0000 |
| Class: 5 | 1         | 0.9955 | 0.9977 |
| Class: 6 | 1         | 1.0000 | 1.0000 |
| Class: 7 | 1         | 1.0000 | 1.0000 |
| Class: 8 | 1         | 1.0000 | 1.0000 |
| Class: 9 | 1         | 1.0000 | 1.0000 |

Evaluate feature importance.

```
importance_matrix <- xgb.importance(feature_names = colnames(X_test_a), model = model)

# Plot top 15 features
xgb.plot.importance(importance_matrix, top_n = 15, main = "Top 15 Important Features")
```

## Top 15 Important Features



## Conclusion

The integration of domain-specific feature engineering is extremely crucial to XGboost in this task. XGBosst is not that capable as I expected. A better generalizable model can be further explored and discussed in later projects.